EXPERIMENT – 1

1. Write a Java program that implements Quick sort algorithm for sorting a list of names in ascending order

```
class Main {

   static int partition(String[] names, int low, int high) {
      String pivot = names[high];
      int i = low - 1;

      for (int j = low; j <= high - 1; j++) {
         if (names[j].compareTo(pivot) < 0) {
            i++;
            swap(names, i, j);
         }
      }

      swap(names, i + 1, high);
      return i + 1;
   }

   static void swap(String[] names, int i, int j) {
      String temp = names[i];
      names[i] = names[j];
      names[j] = temp;
   }

   static void quickSort(String[] names, int low, int high) {
      if (low < high) {
         int pivot = partition(names, low, high);
         quickSort(names, low, pivot - 1);
         quickSort(names, pivot + 1, high);
      }
   }

   public static void main(String[] args) {
      String[] names = {
         "Anvesh", "Kaushik", "Mahira", "Shweta", "Prachi", "Shubham", "Gurvir", "Ibrahim"
      };

      System.out.println("Before Sorting: ");
      for (int i = 0; i < names.length; i++) {
         System.out.println(names[i]);
      }

      quickSort(names, 0, names.length - 1);

      System.out.println();
      System.out.println("After Sorting: ");
```

```
        for (int i = 0; i < names.length; i++) {
            System.out.println(names[i]);
        }
    }
}
```

OUTPUT:

```
Before Sorting:
Anvesh
Kaushik
Mahira
Shweta
Prachi
Shubham
Gurvir
Ibrahim

After Sorting:
Anvesh
Gurvir
Ibrahim
Kaushik
Mahira
Prachi
Shubham
Shweta
```

Figure 1: Output of program 1

EXPERIMENT – 2

2. Write a Java program that implements Bubble sort algorithm for sorting in descending order and also shows the number of interchanges occurred for the given set of integers.

```
public class BubbleSort {
    static void sort(int[] a) {
        int interchange = 0;

        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] < a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                    interchange++;
                }
            }
        }

        System.out.println("Number of interchanges: " + interchange);
    }

    public static void main(String[] args) {
        int[] a = {10, 5, 20, 8, 15};

        System.out.println("Array before sorting:");
        for (int i = 0; i < a.length; i++) {
            System.out.print(a[i] + " ");
        }
        System.out.println();

        sort(a);

        System.out.println("Array after sorting:");
        for (int i = 0; i < a.length; i++) {
            System.out.print(a[i] + " ");
        }
    }
}
```

OUTPUT:

```
Array before sorting:
10 5 20 8 15
Number of interchanges: 6
Array after sorting:
20 15 10 8 5
```

Figure 2: Output of program 2

EXPERIMENT – 3

3. Write a Java program that prompts the user for an integer and then prints out all the prime numbers up to that Integer.

```
import java.util.Scanner;

class PrimeNumbers {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a number: ");
      int num = sc.nextInt();

      System.out.println("Prime numbers from 1 to " + num + " are:");

      for (int i = 2; i <= num; i++) {
         boolean isPrime = true;

         for (int j = 2; j * j <= i; j++) {
            if (i % j == 0) {
               isPrime = false;
               break;
            }
         }

         if (isPrime) {
            System.out.print(i + " ");
         }
      }

      sc.close();
   }
}
```

OUTPUT:

```
Enter a number: 42
Prime numbers from 1 to 42 are:
2 3 5 7 11 13 17 19 23 29 31 37 41
```

Figure 3: Output of program 3

EXPERIMENT – 4

4. Write a Java program that checks whether a given string is a palindrome or not. Ex: MADAM is a palindrome.

```
import java.util.Scanner;

class Palindrome {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a string: ");
      String str = sc.next().toLowerCase();

      boolean isPalindrome = true;
      int j = str.length() - 1;
      for(int i = 0; i < str.length() / 2; i++) {
         if(str.charAt(i) != str.charAt(j--)) {
            isPalindrome = false;
            break;
         }
      }

      if(isPalindrome) {
         System.out.println(str + " is a palindrome.");
      } else {
         System.out.println(str + " is not a palindrome");
      }

      sc.close();
   }
}
```

OUTPUT:

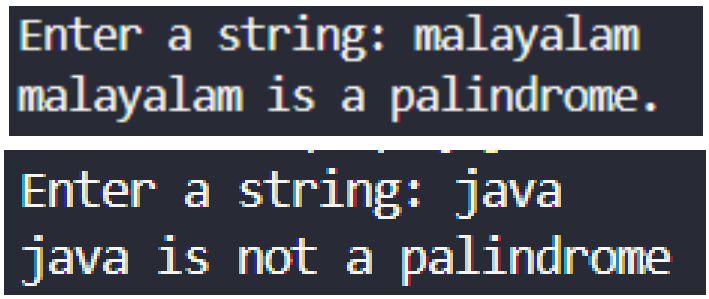


Figure 4: Output of program 4

EXPERIMENT – 5

5. Write a Java program that works as a simple calculator. Use a grid layout to arrange buttons for the digits and for the +, -, *, % operations. Add a text field to display the result. Handle any possible exceptions like divided by zero.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.JOptionPane;

public class Calculator implements ActionListener{
    Frame frame;
    TextField display;
    Button[] digitButtons = new Button[10];
    Button add, subtract, multiply, divide, clear, modulo, equalTo;
    double num1, num2, result;
    char operator;
    boolean calculationDone = false;

    public Calculator() {
        frame = new Frame("Calculator");
        frame.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                frame.dispose();
                System.exit(0);
            }
        });

        display = new TextField();
        display.setEditable(false);
        display.setFont(new Font("Arial", Font.BOLD, 25));
        frame.add(display, BorderLayout.NORTH);

        Panel panel = new Panel();
        panel.setLayout(new GridLayout(4, 4));
        for (int i = 0; i <= 9; i++) {
            digitButtons[i] = new Button(String.valueOf(i));
            digitButtons[i].addActionListener(this);
        }

        add = new Button("+");
        subtract = new Button("-");
        multiply = new Button("*");
        divide = new Button("/");
        modulo = new Button("%");
        equalTo = new Button("=");
        clear = new Button("C");

        add.addActionListener(this);
```

```
        subtract.addActionListener(this);
        multiply.addActionListener(this);
        divide.addActionListener(this);
        modulo.addActionListener(this);
        equalTo.addActionListener(this);
        clear.addActionListener(this);

        panel.add(digitButtons[7]);
        panel.add(digitButtons[8]);
        panel.add(digitButtons[9]);
        panel.add(divide);

        panel.add(digitButtons[4]);
        panel.add(digitButtons[5]);
        panel.add(digitButtons[6]);
        panel.add(multiply);

        panel.add(digitButtons[1]);
        panel.add(digitButtons[2]);
        panel.add(digitButtons[3]);
        panel.add(subtract);

        panel.add(digitButtons[0]);
        panel.add(add);
        panel.add(modulo);
        panel.add(equalTo);

        frame.add(panel, BorderLayout.CENTER);

        frame.add(clear, BorderLayout.SOUTH);

        frame.setSize(400, 400);
        frame.setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        String command = e.getActionCommand();

        try {
            if(command.charAt(0) >= '0' && command.charAt(0) <= '9') {
                if (calculationDone) {
                    display.setText("");
                    calculationDone = false;
                    operator = '\0';
                }
                display.setText(display.getText() + command);
            }
```

```
        else if (command.equals("C")) {
            display.setText("");
            num1 = 0;
            num2 = 0;
            result = 0;
            operator = '\0';
            calculationDone = false;
        }

        else if (
            command.equals("/") ||
            command.equals("*") ||
            command.equals("+") ||
            command.equals("-") ||
            command.equals("%")
        ) {
            if (display.getText().isEmpty()) {
                    throw new NumberFormatException("Enter a number before choosing an
operator");
            }

            num1 = Double.parseDouble(display.getText());
            operator = command.charAt(0);
            calculationDone = false;
            display.setText("");
        }

        else if (command.equals("=")) {
            if (operator == '\0') {
                throw new NumberFormatException("Choose an operator");
            }

            if (!calculationDone && display.getText().isEmpty()) {
                throw new NumberFormatException("Enter some number first");
            }

            if (!calculationDone) {
                num2 = Double.parseDouble(display.getText());
            }

            switch(operator) {
                case '+':
                    result = num1 + num2;
                    break;
                case '-':
                    result = num1 - num2;
                    break;
                case '*':
                    result = num1 * num2;
```

```
                break;
            case '/':
                if(num2 == 0) {
                    throw new ArithmeticException("Cannot divide by zero");
                }
                result = num1 / num2;
                break;
            case '%':
                if(num2 == 0) {
                    throw new ArithmeticException("Cannot divide by zero");
                }
                result = num1 % num2;
                break;
        }

        display.setText(String.valueOf(result));
        num1 = result;
        calculationDone = true;
      }

    } catch (ArithmeticException ex) {
        JOptionPane.showMessageDialog(
            frame,
            ex.getMessage(),
            "Arithmetic Error",
            JOptionPane.ERROR_MESSAGE
        );
        display.setText("");
        return;

    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(
            frame,
            ex.getMessage(),
            "Input Error",
            JOptionPane.ERROR_MESSAGE
        );
        return;
    }
  }


  public static void main(String[] args) {
    new Calculator();
  }
}
```

OUTPUT:

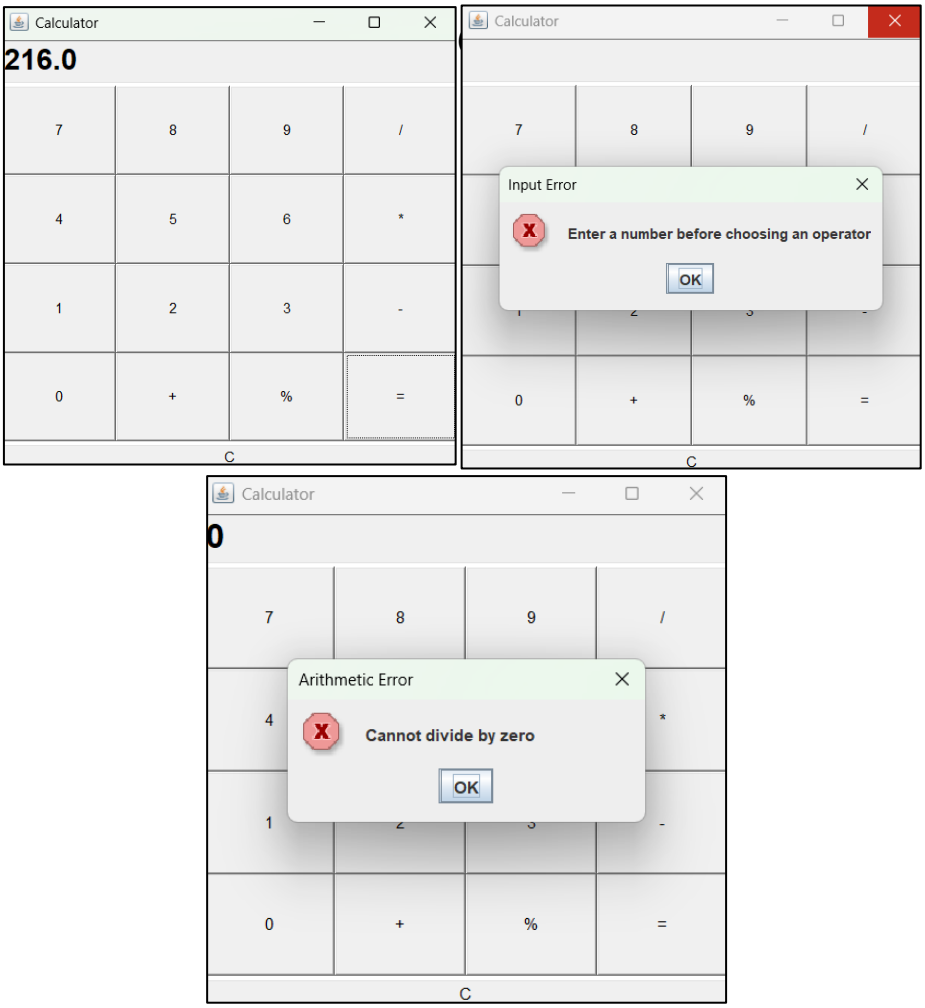


Figure 5: Output of program 5

EXPERIMENT – 6

6. Write a Java program that creates a user interface to perform integer divisions. The user enters two numbers in the text fields, Num1 and Num2. The division of Num1 and Num 2 is displayed in the Result field when the Divide button is clicked. If Num1 or Num2 were not an integer, the program would throw a Number Format Exception. If Num2 were Zero, the program would throw an Arithmetic Exception. Display the exception in a message dialog box.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class IntegerDivision implements ActionListener {
   TextField num1TextField, num2TextField, resultTextField;
   Panel inputPanel, buttonPanel;
   Button integerDivision, equalsTo;
   int num1, num2, result;
   Frame frame;

   public IntegerDivision() {
      frame = new Frame("Integer Division");
      frame.addWindowListener(new WindowAdapter() {
         @Override
         public void windowClosing(WindowEvent e) {
            frame.dispose();
            System.exit(0);
         }
      });

      inputPanel = new Panel();
      inputPanel.setLayout(new GridLayout(1, 3));

      num1TextField = new TextField();
      num1TextField.setFont(new Font("Arial", Font.BOLD, 25));
      inputPanel.add(num1TextField);

      integerDivision = new Button("//");
      inputPanel.add(integerDivision);
      integerDivision.addActionListener(this);

      num2TextField = new TextField();
      num2TextField.setFont(new Font("Arial", Font.BOLD, 25));
      inputPanel.add(num2TextField);

      frame.add(inputPanel, BorderLayout.NORTH);

      buttonPanel = new Panel(new FlowLayout(FlowLayout.CENTER));
      equalsTo = new Button("=");
```

```
        equalsTo.setFont(new Font("Arial", Font.BOLD, 20));
        equalsTo.setPreferredSize(new Dimension(80, 40));
        buttonPanel.add(equalsTo);
        frame.add(buttonPanel, BorderLayout.CENTER);
        equalsTo.addActionListener(this);

        resultTextField = new TextField();
        resultTextField.setEditable(false);
        resultTextField.setFont(new Font("Arial", Font.BOLD, 25));
        frame.add(resultTextField, BorderLayout.SOUTH);

        frame.setSize(400, 170);
        frame.setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        String command = e.getActionCommand();

        if (command.equals("=")) {
            try {
                num1 = Integer.parseInt(num1TextField.getText().trim());
                num2 = Integer.parseInt(num2TextField.getText().trim());

                if (num2 == 0) {
                    throw new ArithmeticException(
                        "Cannot Divide by Zero."
                    );
                }

                result = num1 / num2;
                resultTextField.setText(String.valueOf(result));

            } catch (ArithmeticException ex) {
                JOptionPane.showMessageDialog(
                    frame,
                    ex.getMessage(),
                    "Arithmetic Error",
                    JOptionPane.ERROR_MESSAGE
                );
                resultTextField.setText("");

            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(
                    frame,
                    "Please enter valid integers.",
                    "Input Error",
                    JOptionPane.ERROR_MESSAGE
                );
```

```
                resultTextField.setText("");
            }
        }
    }

    public static void main(String[] args) {
        new IntegerDivision();
    }
}
```

OUTPUT:

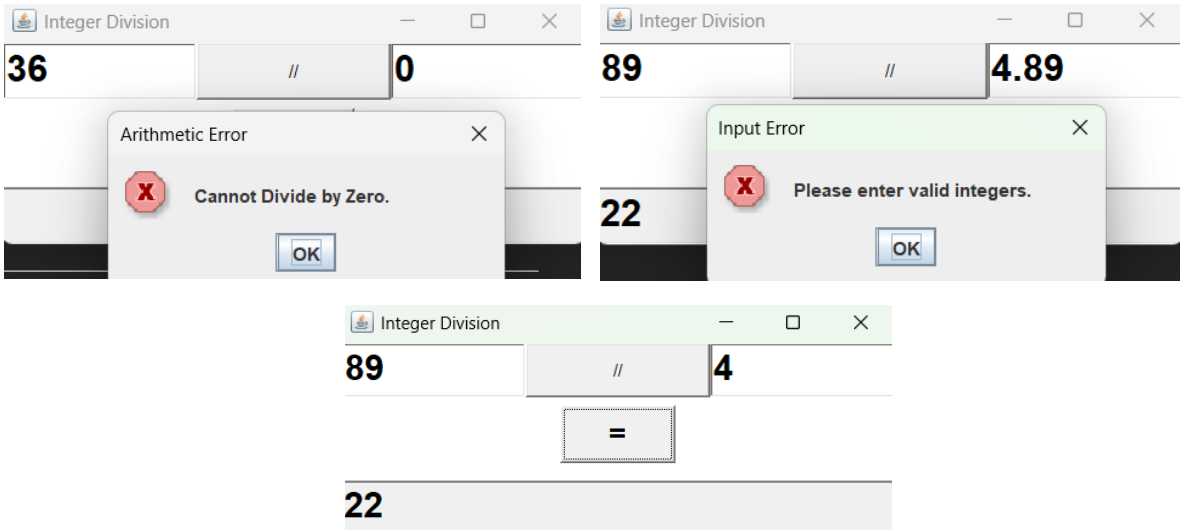


Figure 6: Output of program 6

EXPERIMENT – 7

7. Write a Java program for the following: Create a doubly linked list of elements. Delete a given element from the above list. Display the contents of the list after deletion.

```
import java.util.LinkedList;

public class DoublyLinkedList {
   public static void main(String[] args) {
      LinkedList<String> list = new LinkedList<>();

      // Insert elements
      list.add("Apple");
      list.add("Banana");
      list.add("Mango");
      list.addFirst("Orange");
      list.addLast("Grapes");

      System.out.println("Original List: " + list);

      // Insert at a specific index
      list.add(2, "Pineapple");
      System.out.println("After insertion: " + list);

      // Display first and last elements
      System.out.println("First Element: " + list.getFirst());
      System.out.println("Last Element: " + list.getLast());

      // Display in reverse order
      System.out.println("Reverse List: " + list.reversed());

      // Delete an element by value
      list.remove("Banana");
      System.out.println("After removing Banana: " + list);

      // Delete an element by index
      list.remove(2);
      System.out.println("After removing index 2: " + list);

      // Delete first and last elements
      list.removeFirst();
      list.removeLast();
      System.out.println("After removing first and last: " + list);

      // Check whether an element exists
      System.out.println("Contains Mango? " + list.contains("Mango"));

      // Display size
      System.out.println("Size: " + list.size());
```

```
        // Traverse the list
        System.out.println("Traversing the list:");
        for (String fruit : list) {
            System.out.println(fruit);
        }
    }
}
```

OUTPUT:

```
Original List: [Orange, Apple, Banana, Mango, Grapes]
After insertion: [Orange, Apple, Pineapple, Banana, Mango, Grapes]
First Element: Orange
Last Element: Grapes
Reverse List: [Grapes, Mango, Banana, Pineapple, Apple, Orange]
After removing Banana: [Orange, Apple, Pineapple, Mango, Grapes]
After removing index 2: [Orange, Apple, Mango, Grapes]
After removing first and last: [Apple, Mango]
Contains Mango? true
Size: 2
Traversing the list:
Apple
Mango
```

Figure 7: Output of program 7

EXPERIMENT – 8

8.a: Develop an applet in Java that displays a simple message.

Traditional Code (Java 9 or earlier):

```
import java.applet.Applet;
import java.awt.Graphics;

public class MessageApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello! Welcome to Java Applets.", 50, 100);
    }
}
```

Modern JDKs do not support Applet, using Java AWT

```
import java.awt.Frame;
import java.awt.Graphics;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

public class MessageApplet extends Frame {
    MessageApplet() {
        setTitle("Simple Java Message");
        setSize(400, 200);
        setVisible(true);

        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                dispose();
            }
        });
    }

    public void paint(Graphics g) {
        super.paint(g);
        g.drawString("Hello! Welcome to Java Applets.", 100, 100);
    }

    public static void main(String[] args) {
        new MessageApplet();
    }
}
```

OUTPUT:

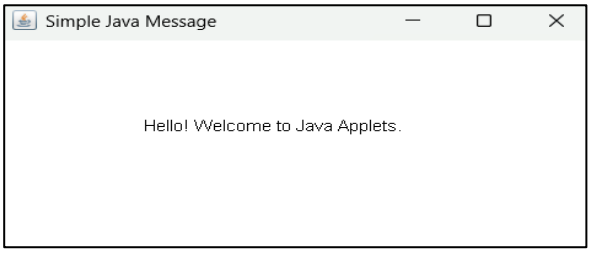


Figure 8: Output of program 8.a

8.b: Develop an applet in Java that receives an integer in one text field, and computes its factorial Value and returns it in another text field, when the button named “Compute” is clicked.